Planning with Monte Carlo tree search

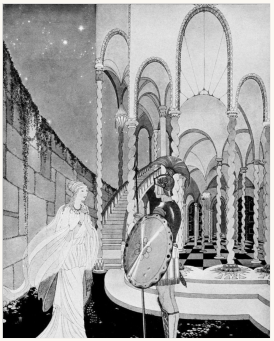
Imagine many endings, then choose the first step

Carlos Cotrini

ETH Zurich

October 2026

The Minotaur, and the two sails



Virginia Frances Sterrett, *Tanglewood Tales*, 1921.

A bull-faced monster lived in a labyrinth on Crete. Nobody who went in found the way out.

Each year Athens sent it seven young men and seven young women. Theseus was among them.

The ship had a black sail. His father said: if you come back alive, spread white sails.

Apollodorus, *Library* 3.1.4, 3.15.8, *Epitome* 1.7 (Frazer, 1921).

The thread, and the black sail



Theseus and Ariadne, Pompeii (drawing by G. Discanno).

Ariadne asked the right question, how to get out again, and gave Theseus a ball of thread.

He tied one end to the door, killed the monster and followed the thread out.

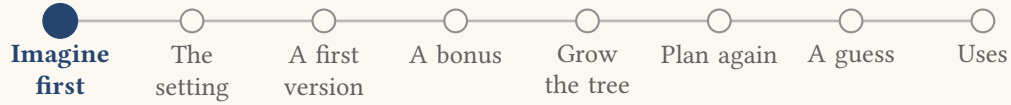
Then he sailed home and forgot the white sails.

His father Aegeus saw the black sail, supposed him dead, and threw himself down.

Apollodorus, *Epitome* 1.8 to 1.10 (Frazer, 1921).

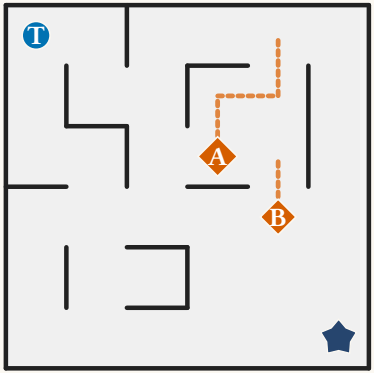
Imagine before you step

A labyrinth with two Medusas, and a habit that programs share.



Theseus and the two Medusas

In our version, two Medusas patrol the labyrinth.



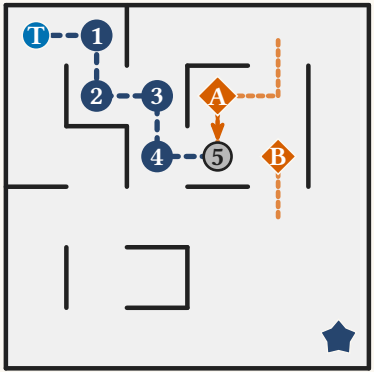
1 Theseus, **T**, wants the lair, the star.

2 Medusas **A** and **B** walk dotted rounds for ever.

3 Anyone in her cell, or the cell she enters next, turns to stone.

He can imagine the ending

A wrong step is final, but an imagined walk costs nothing.



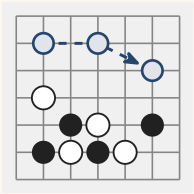
1 On paper the shortest route is 10 steps.

2 In his head he walks it, turn by turn.

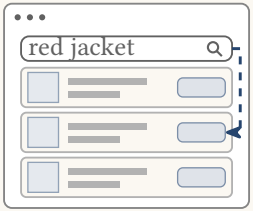
3 At turn 5 Medusa **A** reaches him: stone. Nothing is lost.

The same habit runs inside programs you know

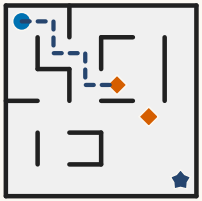
Some Go programs and web agents imagine many endings first.



a program that plays Go



a program that shops for you

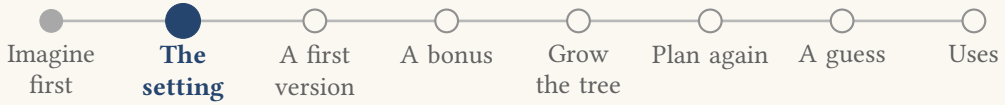


Theseus

Dashed: the endings each one imagines first.

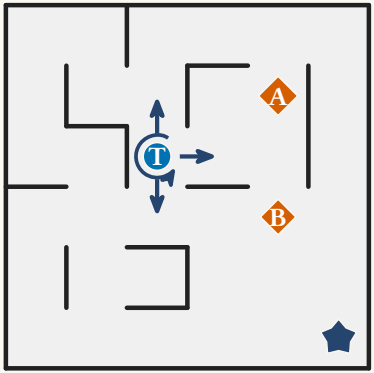
The setting

States, actions, returns and values, told on the labyrinth.



State and actions

State: a cell and a turn. Action: stay, or step.



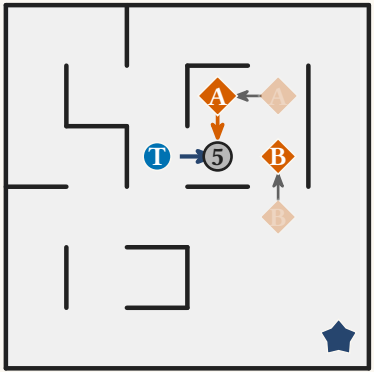
1 Here: the **T**'s cell, turn 4.

2 The Medusas keep a timetable: another turn, another state.

3 Here: four actions, stay or step up, right or down.

Transitions

Asking the world what follows an action costs nothing.



1 He asks about one action: step right.

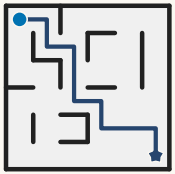
2 The answer: the next state, turn 5.

3 It also says if the run is over: stone.

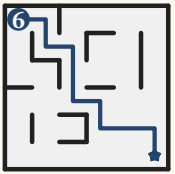
Reward and return

A run pays a return G, from 0 to 1: the sooner at the lair, the more.

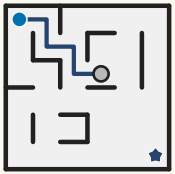
Reward: a turn pays 0, except reaching the lair on turn t: 0.97^t.
Return G**:** the rewards added up.



at the lair
on turn 10
$G = 0.97^{10} \approx 0.74$



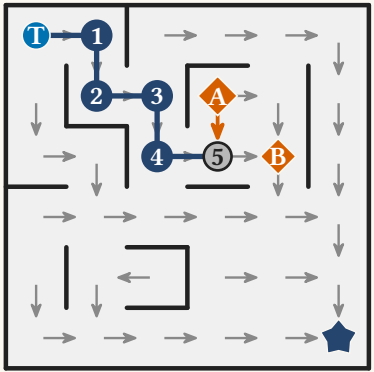
waits 6 turns, then at
the lair on turn 16
$G = 0.97^{16} \approx 0.61$



turned to stone
on turn 5
$G = 0$

Policy and trajectory

A policy picks an action in every state.



1 This one steps to the cell nearest the lair.

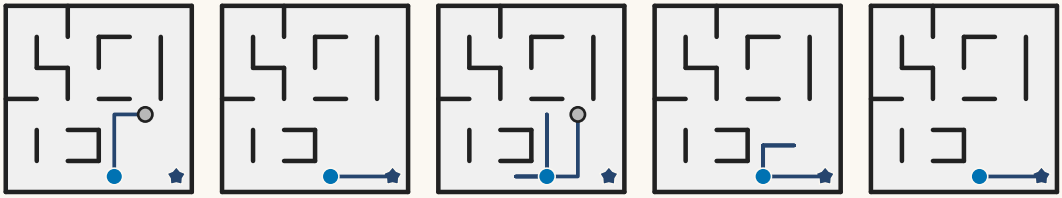
2 From the door it gives a trajectory.

3 It ends in stone on turn 5: $G = 0$.

The value of a state

A state's value $V(s)$ is the average G over many runs.

Five random runs from the blue dot, on turn 6.

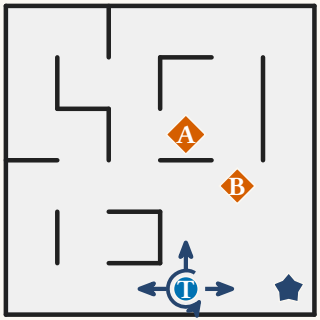


stone on turn 9	lair on turn 10	stone on turn 15	lair on turn 18	lair on turn 8
$G = 0.00$	$G = 0.74$	$G = 0.00$	$G = 0.58$	$G = 0.78$

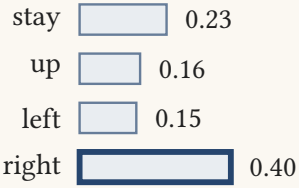
The five average 0.42, 10,000 runs 0.24: the $V(s)$ of the policy-gradient lectures.

The goal

Choose the action with the largest average G.



average G of 10,000 random runs after each action



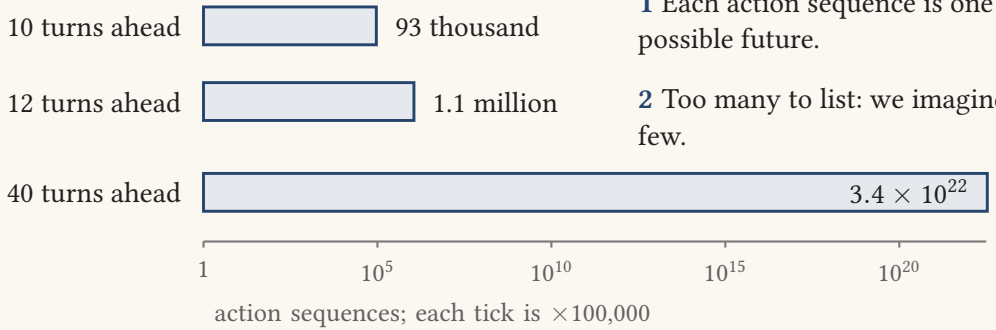
1 Per action: imagine many runs, average their G.

2 Play the largest: here, right.

3 No training, only simulation.

Too many futures

The futures from the door explode with depth.

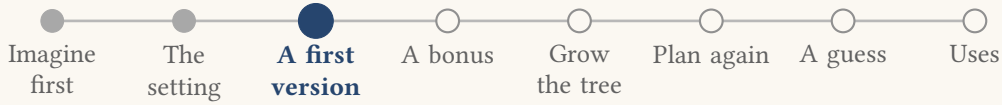


1 Each action sequence is one possible future.

2 Too many to list: we imagine a few.

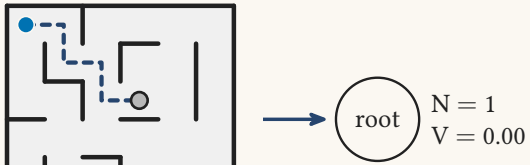
A first version

A tree of passes, and the move it chooses.



A counter on every state

Each state carries N, its passes, and V, their average G.



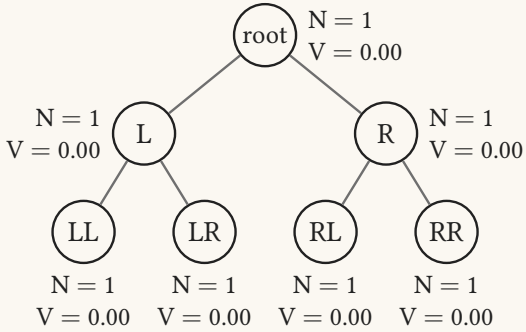
a state: the door, on turn 0

1 A pass is one imagined run through it, here ending in stone: $G = 0$.

2 Born with $N = 1$ and $V = 0$: one imagined visit that paid nothing.

A tree of depth d

The first version: a node for every state up to d steps ahead.



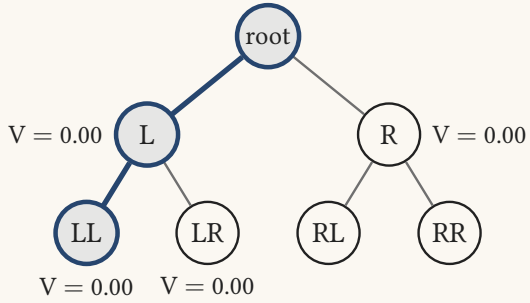
1 A toy labyrinth: three forks. The door is the root.

2 With $d = 2$: 7 nodes.

3 Each starts at $N = 1$, $V = 0$.

A pass, step 1: go down

A pass goes down, into the child with the largest V.

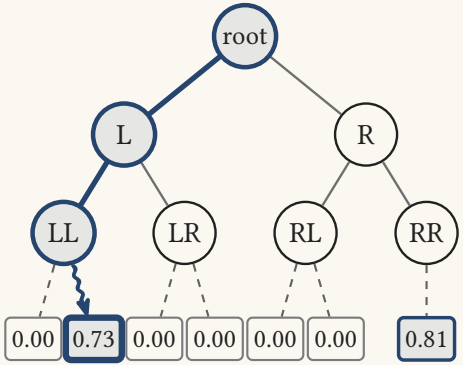


1 $V = 0$ twice: a tie, so the left child.

2 Again, and it stops at LL: the tree ends.

A pass, step 2: a rollout

A rollout plays at random to the end and reads off G.

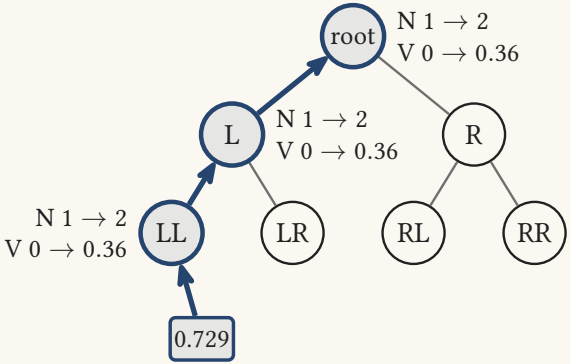


1 In the toy a lair on turn t pays 0.9^t, any other end 0.

2 $G = 0.9^3 = 0.729$. A rollout adds no nodes.

A pass, step 3: walk back up

Each node on the path counts the visit and moves V toward G.



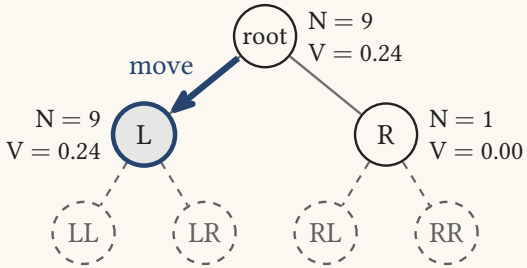
1 $G = 0.729$ goes back up the path.

2 Each node: $N \leftarrow N + 1$, $V \leftarrow V + (G - V)/N$. At LL, $V = 0.00 + (0.729 - 0.00)/2 \approx 0.36$.

3 Same at L and the root; the others are untouched.

M passes, then one move

After M passes, he moves to the most visited child.

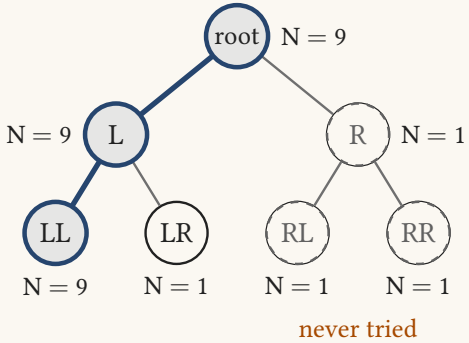


1 Left has $N = 9$: 8 passes and its birth visit. Right still has $N = 1$.

2 He moves into it: left.

When the first success takes over

With ties to the first child, the first success keeps every pass.



1 All 8 went left: any V above 0 beats an untried 0.

2 Right was never tried, yet it averages 0.41 against 0.18 for left.

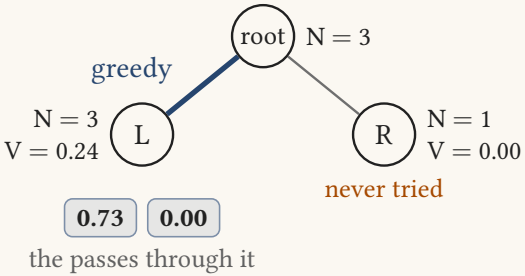
A bonus for the barely tried

Why the first version sticks, and how UCT repairs it.



A child that paid once

A child that paid once beats one nobody tried, and greedy never goes back.

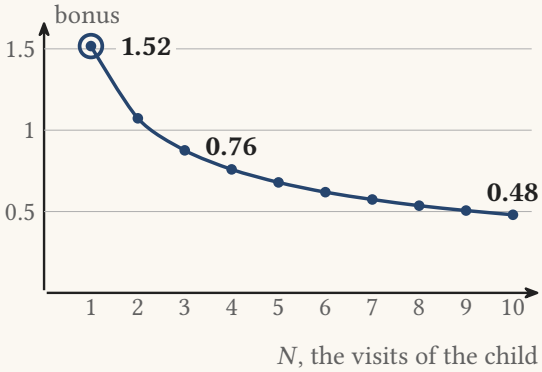


1 Two passes so far, both through the left child.

2 Only a pass through the right could change its V; greedy sends none.

A bonus for the barely tried

Add to V a bonus that shrinks as the child's visits grow.



1 For a parent visited 10 times.

2 An untried child, $N = 1$, gets the largest: 1.52.

3 It shrinks: 0.76 at $N = 4$, 0.48 at $N = 10$.

UCT

A pass goes into the child with the largest score.

UCT: Upper Confidence bounds applied to Trees.

$$\mathrm{UCT}(s) \;=\; V(s) \;+\; w \cdot \mathrm{bonus}(s)$$

$$\mathrm{bonus}(s) \;=\; \sqrt{\frac{\ln N(\mathrm{parent}(s))}{N(s)}}$$

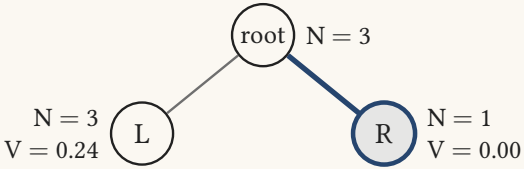
ln: natural logarithm.

1. w is a weight we choose.
2. It is large when $N(s)$ is small next to its parent's.
3. Each pass that skips s raises it, so s gets its turn.

Only the descent changes.

One choice, worked out

Left has the larger V, the untried right child the larger score.



	bonus	score = V + bonus
L	$\sqrt{\ln 3 \,/\, 3} = 0.61$	$0.24 + 0.61 = \mathbf{0.85}$
R	$\sqrt{\ln 3 \,/\, 1} = 1.05$	$0.00 + 1.05 = \mathbf{1.05}$

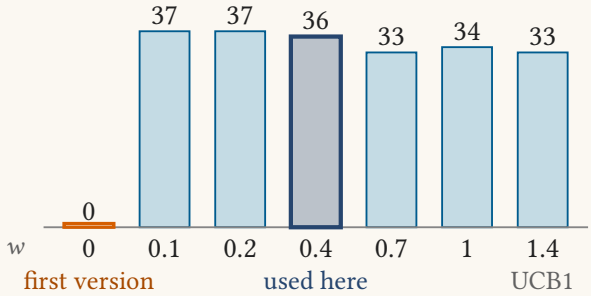
1 Pass 3, with $w = 1$.

2 Right scores 1.05 against 0.85, so pass 3 goes right. Greedy would go left.

The size of w

w, the exploration weight, sets how much the unknown counts.

games won of 40; full tree of depth 2; 100 passes a move



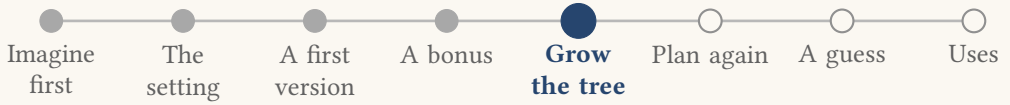
1 The first version, $w = 0$, wins none; any other w wins 33 to 37.

2 UCB1 uses $\sqrt{2} \approx 1.4$; we use 0.4.

Auer, Cesa-Bianchi, Fischer (2002), p. 245: UCB1.

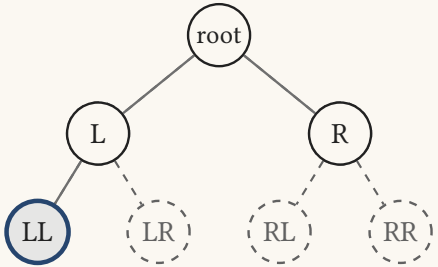
Grow the tree where passes go

One node a pass: select, expand, simulate, back up.



The tree grows only where passes go

A full tree is mostly never visited: grow one node per pass.



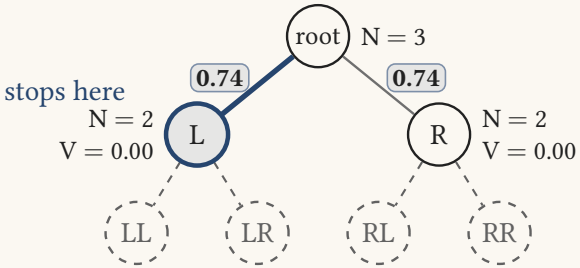
1 At depth 10 its last row has 93 thousand nodes.

2 Pass 1 adds L, pass 2 R, pass 3 LL.

3 Dashed: not built yet.

Where a pass stops

A pass goes down until a node has an untried action.

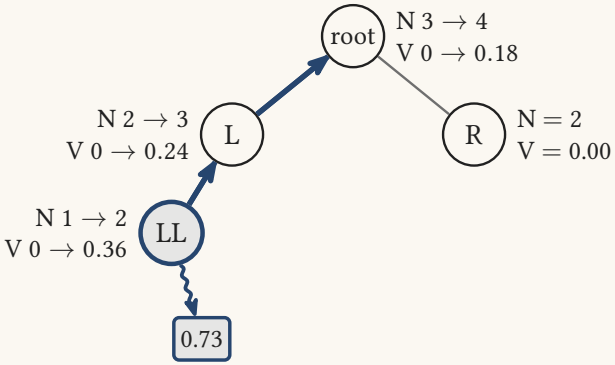


1 L and R exist; the dashed nodes are untried actions.

2 The larger UCT wins: a tie at 0.74, so L.

The new node is the start

The new child starts the rollout and the walk back up.



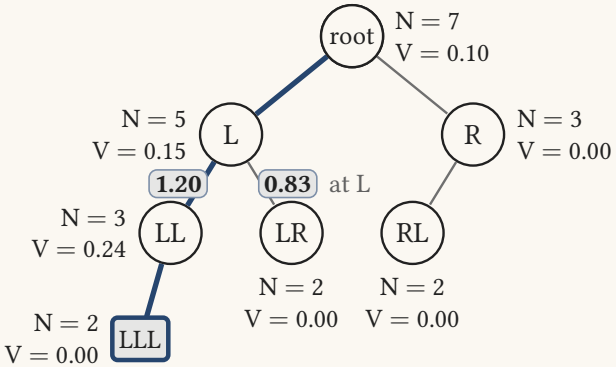
1 It adds LL, born with $N = 1$, $V = 0$.

2 The rollout starts at LL and reaches the lair: $G = 0.73$.

3 LL, L and the root count it; R is untouched.

Six passes on the toy

Six passes grow the tree to 7 nodes, UCT at $w = 1$.

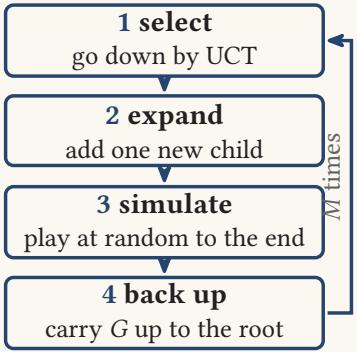


pass	adds	paid	UCT of L / R
1	L	0	*untried*
2	R	0	*untried*
3	LL	0.73	**0.74** / 0.74
4	LR	0	**0.92** / 0.83
5	RL	0	0.82 / **0.90**
6	LLL	0	**0.85** / 0.77

Pass 4 stops at L: LR untried.
Pass 5: R has the lower V but the higher UCT.

The four phases

Every pass: select, expand, simulate, back up.



`mcts_move(state, n_attempts, w, rng)`

```
root = Node(state, None)
for i in range(n_attempts):
1   leaf = descend(root, w)
2   leaf = expand(leaf)
3   r = rollout_reward(leaf.state)
4   backprop(leaf, r)
return most_visited_child(root)
```

M is `n_attempts` in the code.

The search in code: select and expand

One function walks down the tree, the other adds a node.

1
```
def descend(s, w):
    if len(s.untried) == 0 and not game_over(s.state):
        best = best_child_by_uct(s, w)                    # largest UCT
        return descend(best, w)
    else:
        return s                                          # stop: untried action, or run over
```

2
```
def expand(s):
    if game_over(s.state):
        return s
    action = s.untried.pop(0)                             # first untried action, removed
    new_state = env.step(s.state, action)
    child = Node(new_state, s)
    s.children.append(child)
    return child                                          # start of rollout and back up
```

The search in code: simulate and back up

One function reads off G, the other carries it up the path.

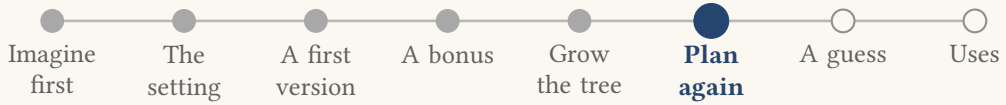
3
```
def rollout_reward(state, rng):
    final_state = rollout(state, rng)          # random play to the end
    return outcome(final_state)                # its return, G
```

4
```
def backprop(node, r):
    node.N = node.N + 1                        # one more visit
    node.V = (node.V * (node.N - 1) + r) / node.N
    if node.parent is not None:
        backprop(node.parent, r)               # on to the parent
```

That is the running mean: $V \leftarrow V + (r - V)/N$.

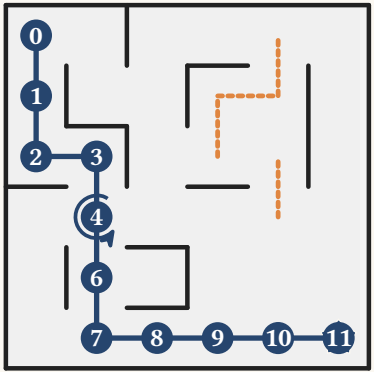
Planning is repeated

One search buys one step; then we search again.



Plan, step, plan again

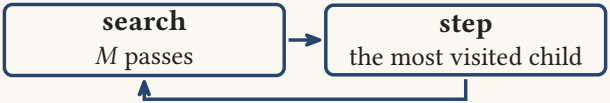
One search buys one step; then we search again.



1 At the door, 100 passes: stay $N = 33$, right 32, down 38. He steps down.

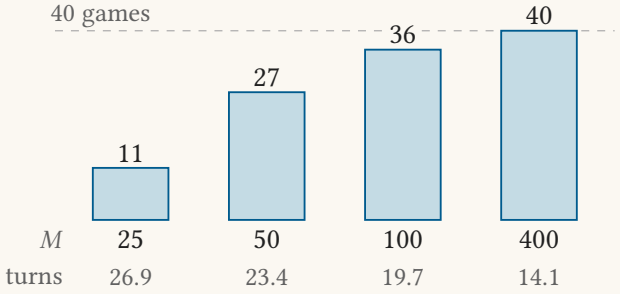
2 The Medusas move: search again from here.

3 11 steps, 11 searches.



How many passes

Wins climb with M: 11 of 40 games at $M = 25$, 40 at $M = 400$.



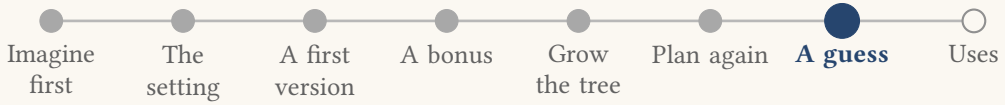
Turns of won games; best plan: 10.

1 Won games get shorter too.

2 A game at $M = 400$ costs about 5.6 thousand random runs.

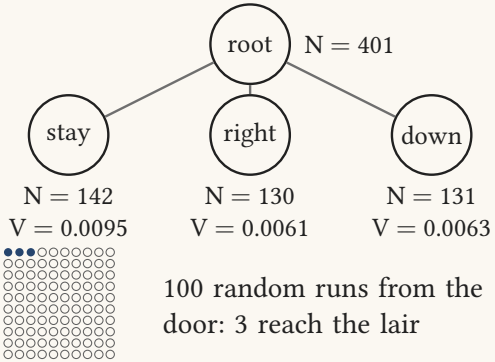
Start from a guess

When random rollouts know nothing, a guess helps.



What the rollouts know

Rollouts that pay nothing make the first moves look alike.



100 random runs from the door: 3 reach the lair

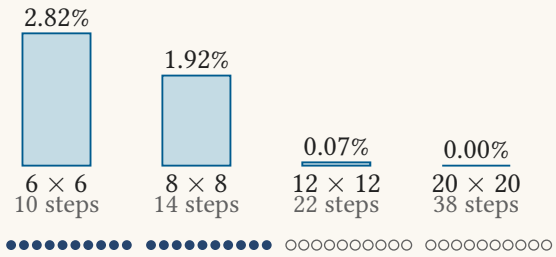
1 From the door, 97.2% of random runs pay nothing.

2 After 400 passes their V differ by only 0.0035.

3 The bonus, about 0.08, is 24 times that, so counts stay level.

The wall

On larger labyrinths random rollouts stop reaching the lair.



Bars: walks that reach the lair.
Dots: games won of 10.

1 Bigger labyrinths, same search: $M = 400$.

2 The share falls: 2.82%, 0.07%, then 0.00%.

3 Wins fall too: only zeros to average.

Start from a guess

A new node can start from a guess V_0.

a new node: the move down



imagined real rollouts

$N = 3$, $V = (0.74 + 0 + 0)/3 = 0.25$

the guess: what it pays with no Medusas, $0.97^{1+9} = 0.74$

As before: $N \leftarrow N + 1$, $V \leftarrow V + (G - V)/N$.

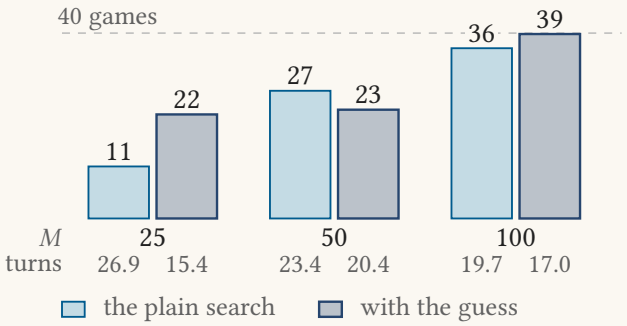
1 Until now: one imagined visit that paid nothing.

2 Now: n_0 imagined visits, each paying V_0.

3 Real rollouts that pay nothing pull it down.

A guess lifts a starved search

A guess that ignores the Medusas lifts $M = 25$ from 11 to 22 wins of 40.



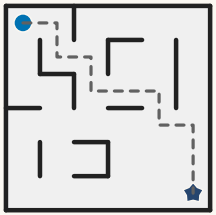
1 Plain: 11, 27 and 36 wins.

2 With the guess: 22, 23, 39: 11 more, 4 fewer, 3 more.

3 Turns fall too.

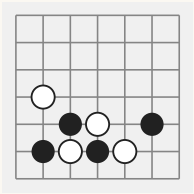
Where guesses come from

A rule, a network or a language model can supply V_0.



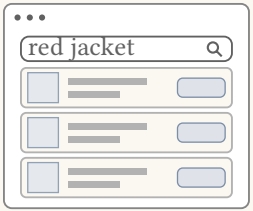
a rule: the walking distance

$V_0 = 0.97^{t+d}$



a trained network

its value



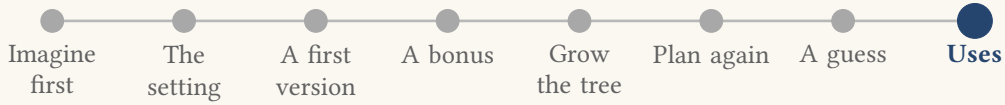
a language model

its score

The guess starts the node; rollouts correct it.

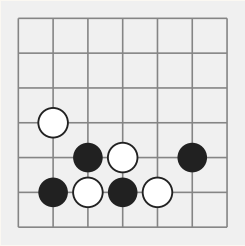
Where it is used

AlphaGo and language-model agents run a version of this loop.



AlphaGo, in the four words

The same four words, for the game of Go.



a position

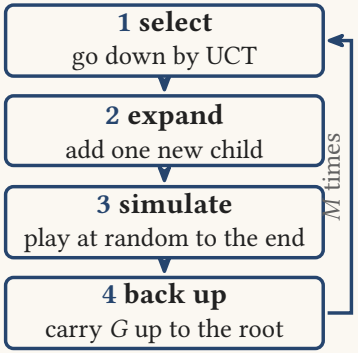
state	the board position
action	a legal move
transition	the rules of Go
return	win or loss

About 250 moves a position, 150 moves a game.

Silver et al., *Nature* 529, 484 to 489 (2016).

What AlphaGo adds to the loop

AlphaGo keeps the loop and the most visited move; networks feed it.



1. Priors from a policy network scale the bonus.

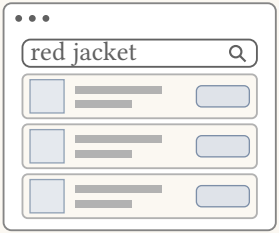
3. Network value v and fast rollout z score the leaf.

   $V = (1 - \lambda)\, v + \lambda\, z$, with $\lambda = 0.5$

4. Two players: one's win is the other's loss.

LATS, in the four words

The same four words, for a language-model agent.



a task on the web

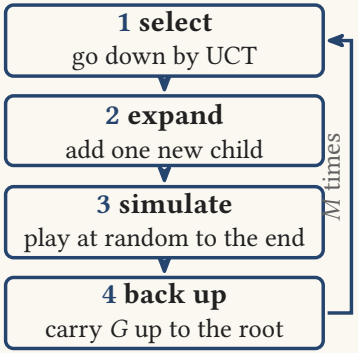
state	the task so far, as text
action	$n = 5$ steps written by the model
transition	the site answers; replay to go back
return	the task's result

Any state is restored by pasting its text back.

Zhou et al., ICML 2024 (PMLR 235).

What LATS adds to the loop

LATS puts a language model in the loop, and trains nothing.



2. The model proposes 5 actions and scores each.
3. The rollout follows the scores.
4. A failed run leaves a written reflection.

Three worlds, one loop

Actions and values come from different places; the loop stays.

	Theseus	AlphaGo	LATS
actions	every legal step	legal moves, with network priors	$n = 5$ steps written by the model
values	random rollouts, or a guess	a value network and a fast rollout	the model's score and self-consistency
transition	the labyrinth's step function	the rules of Go	the site, reset by replaying the text
training	none	networks trained on games	none

What the method needs

Three things the method needs from the world.

1. **A world you can reset**

   Asking what follows an action costs nothing.

2. **A cheap rollout, or a guess**

   Each pass must be cheap and carry some signal.

3. **A budget of passes**

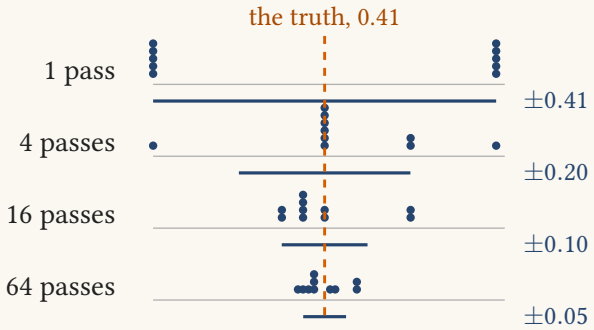
   More passes buy better moves.

Appendix

Extras for questions: where the bonus comes from, and more.

Where the bonus comes from

An average of N passes strays from the truth like $1/\sqrt{N}$.



Auer, Cesa-Bianchi, Fischer (2002), pp. 237, 245.

1 Right fork: a pass pays 0.81 or 0, equally likely. Each row: 10 averages of N passes.

2 More passes crowd round the truth.

3 The bar: how far an average typically strays, half as far when N quadruples.

From a confidence bound to the bonus

A confidence bound on an average, rewritten, is the bonus of UCT.

1. n plays of a child, each G in $[0, 1]$: average $\bar{G}$, true mean μ.
2. Hoeffding: $\Pr(\mu \geq \bar{G} + \varepsilon) \leq e^{-2n\varepsilon^2}$.
3. Set the right side to δ: $\varepsilon = \sqrt{\ln(1/\delta)/(2n)}$; for $\delta = 0.05$, $n = 10$ it is 0.39.
4. Take $\delta = N^{-4}$, N the parent's passes: $\varepsilon = \sqrt{2 \ln N / n}$.
5. Write the constant $\sqrt{2}$ as a weight w we tune: the bonus is $w\sqrt{\ln N / n}$.

Hoeffding (1963); Kocsis and Szepesvári (2006), eqs. 2, 3.

The update, step by step

The update $V \leftarrow V + (G - V)/N$ is the running mean, rewritten.

1. After $N - 1$ visits the node holds V_{old}: its G's add up to $V_{\text{old}}(N - 1)$.
2. A new pass adds G and one visit: $V = \dfrac{V_{\text{old}}(N - 1) + G}{N}$.
3. Rearranged: $V = V_{\text{old}} + \dfrac{G - V_{\text{old}}}{N}$.

Node L of the toy: pass 1 pays 0.729, so $N = 2$ and $V = 0 + (0.729 - 0)/2 = 0.36$. Pass 2 pays 0, so $N = 3$ and $V = 0.36 + (0 - 0.36)/3 = 0.24$.

Born at $N = 1$

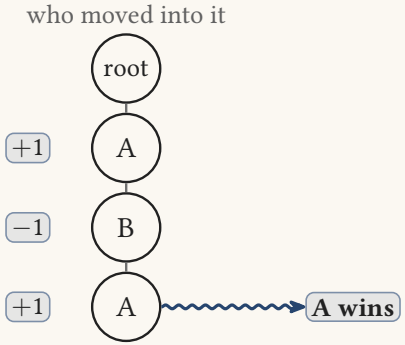
The birth visit is one imagined pass that paid nothing.

1. It adds one to the count and 0 to the sum.
2. After k passes: $N = k + 1$ and $V = \dfrac{G_1 + \cdots + G_k}{k + 1}$.
3. Node L after two passes: $(0.729 + 0)/3 = 0.24$; the plain average is 0.36.

Starting at $N = 0$ divides the bonus by 0, so each child needs a first try.

Two players, one tree

Each node is valued for the player who moved into it.



1 A's win counts $+1$ at the nodes A moved into.

2 At the nodes B moved into it counts -1.

3 So the sign flips at every step up.

AlphaGo, Methods: $r_1(s) = -r_2(s)$ (Silver et al., 2016).

What UCT is proved to do

The proof gives convergence, and says little about speed or the weight.

Proved

The chance of a wrong move at the root tends to 0, at a polynomial rate.

The proof scales the bonus by the horizon D.

Left open

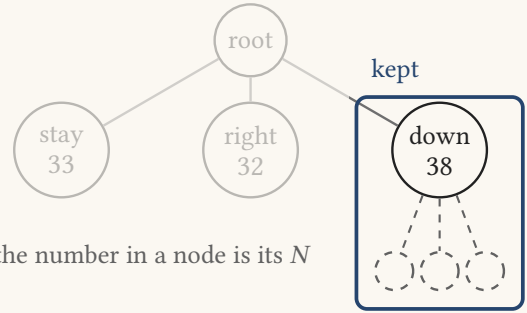
The best w: here we tuned it, 0.4.

Speed: in some trees UCT is over-optimistic, with very poor worst-case regret.

Kocsis and Szepesvári, ECML 2006, Theorem 6. Coquelin and Munos, UAI 2007.

Reusing the subtree

After the step, planners keep what the search already learned.



As in AlphaGo, Methods (Silver et al., 2016).

1 The chosen child, down, becomes the root.

2 Its subtree keeps its counts.

3 Here each step starts from a new tree: simpler.